23-Host容器：Tomcat如何实现热部署和热加载？

从这一期我们开始学习Tomcat的容器模块，来聊一聊各容器组件实现的功能，主要有热部署热加载、类加载机制以及Servlet规范的实现。最后还会谈到Spring Boot是如何与Web容器进行交互的。

今天我们首先来看热部署和热加载。要在运行的过程中升级Web应用，如果你不想重启系统，实现的方式有两种：热加载和热部署。

那如何实现热部署和热加载呢？它们跟类加载机制有关，具体来说就是：

- 热加载的实现方式是Web容器启动一个后台线程，定期检测类文件的变化，如果有变化，就重新加载类，在这个过程中不会清空Session ，一般用在开发环境。
- 热部署原理类似，也是由后台线程定时检测Web应用的变化，但它会重新加载整个Web应用。这种方式会清空Session，比热加载更加干净、彻底，一般用在生产环境。

今天我们来学习一下Tomcat是如何用后台线程来实现热加载和热部署的。Tomcat通过开启后台线程，使得各个层次的容器组件都有机会完成一些周期性任务。我们在实际工作中，往往也需要执行一些周期性的任务，比如监控程序周期性拉取系统的健康状态，就可以借鉴这种设计。

Tomcat的后台线程

要说开启后台线程做周期性的任务，有经验的同学马上会想到线程池中的ScheduledThreadPoolExecutor，它除了具有线程池的功能，还能够执行周期性的任务。Tomcat就是通过它来开启后台线程的：

```
bgFuture = exec.scheduleWithFixedDelay(
                new ContainerBackgroundProcessor(),//要执行的Runnable
                backgroundProcessorDelay, //第一次执行延迟多久
                backgroundProcessorDelay, //之后每次执行间隔多久
                TimeUnit.SECONDS);        //时间单位
```

上面的代码调用了scheduleWithFixedDelay方法，传入了四个参数，第一个参数就是要周期性执行的任务类ContainerBackgroundProcessor，它是一个Runnable，同时也是ContainerBase的内部类，ContainerBase是所有容器组件的基类，我们来回忆一下容器组件有哪些，有Engine、Host、Context和Wrapper等，它们具有父子关系。

ContainerBackgroundProcessor实现

我们接来看ContainerBackgroundProcessor具体是如何实现的。

```
protected class ContainerBackgroundProcessor implements Runnable {

    @Override
    public void run() {
        //请注意这里传入的参数是"宿主类"的实例
        processChildren(ContainerBase.this);
    }
```

仅供个人学习 请勿传播 用户标记 mannix小查理

```
    protected void processChildren(Container container) {
        try {
            //1. 调用当前容器的backgroundProcess方法。
            container.backgroundProcess();

            //2. 遍历所有的子容器，递归调用processChildren，
            //这样当前容器的子孙都会被处理
            Container[] children = container.findChildren();
            for (int i = 0; i < children.length; i++) {
            //这里请你注意，容器基类有个变量叫做backgroundProcessorDelay，如果大于0，表明子容器有自己的后台线程，无需父容
                if (children[i].getBackgroundProcessorDelay() <= 0) {
                    processChildren(children[i]);
                }
            }
        } catch (Throwable t) { ... }
```

上面的代码逻辑也是比较清晰的，首先ContainerBackgroundProcessor是一个Runnable，它需要实现run方法，它的run很简单，就是调用了processChildren方法。这里有个小技巧，它把“宿主类”，也就是**ContainerBase的类实例当成参数传给了run方法。**

而在processChildren方法里，就做了两步：调用当前容器的backgroundProcess方法，以及递归调用子孙的backgroundProcess方法。请你注意backgroundProcess是Container接口中的方法，也就是说所有类型的容器都可以实现这个方法，在这个方法里完成需要周期性执行的任务。

这样的设计意味着什么呢？我们只需要在顶层容器，也就是Engine容器中启动一个后台线程，那么这个线程**不但会执行Engine容器的周期性任务，它还会执行所有子容器的周期性任务。**

backgroundProcess方法

上述代码都是在基类ContainerBase中实现的，那具体容器类需要做什么呢？其实很简单，如果有周期性任务要执行，就实现backgroundProcess方法；如果没有，就重用基类ContainerBase的方法。ContainerBase的backgroundProcess方法实现如下：

```
public void backgroundProcess() {

    //1.执行容器中Cluster组件的周期性任务
    Cluster cluster = getClusterInternal();
    if (cluster != null) {
        cluster.backgroundProcess();
    }

    //2.执行容器中Realm组件的周期性任务
    Realm realm = getRealmInternal();
    if (realm != null) {
        realm.backgroundProcess();
   }

   //3.执行容器中Valve组件的周期性任务
    Valve current = pipeline.getFirst();
    while (current != null) {
       current.backgroundProcess();
       current = current.getNext();
    }
```

```
    //4. 触发容器的"周期事件"，Host容器的监听器HostConfig就靠它来调用
    fireLifecycleEvent(Lifecycle.PERIODIC_EVENT, null);
}
```

从上面的代码可以看到，不仅每个容器可以有周期性任务，每个容器中的其他通用组件，比如跟集群管理有关的Cluster组件、跟安全管理有关的Realm组件都可以有自己的周期性任务。

我在前面的专栏里提到过，容器之间的链式调用是通过Pipeline-Valve机制来实现的，从上面的代码你可以看到容器中的Valve也可以有周期性任务，并且被ContainerBase统一处理。

请你特别注意的是，在backgroundProcess方法的最后，还触发了容器的“周期事件”。我们知道容器的生命周期事件有初始化、启动和停止等，那“周期事件”又是什么呢？它跟生命周期事件一样，是一种扩展机制，你可以这样理解：

又一段时间过去了，容器还活着，你想做点什么吗？如果你想做点什么，就创建一个监听器来监听这个“周期事件”，事件到了我负责调用你的方法。

总之，有了ContainerBase中的后台线程和backgroundProcess方法，各种子容器和通用组件不需要各自弄一个后台线程来处理周期性任务，这样的设计显得优雅和整洁。

Tomcat热加载

有了ContainerBase的周期性任务处理“框架”，作为具体容器子类，只需要实现自己的周期性任务就行。而Tomcat的热加载，就是在Context容器中实现的。Context容器的backgroundProcess方法是这样实现的：

```
public void backgroundProcess() {

    //WebappLoader周期性的检查WEB-INF/classes和WEB-INF/lib目录下的类文件
    Loader loader = getLoader();
    if (loader != null) {
        loader.backgroundProcess();
    }

    //Session管理器周期性的检查是否有过期的Session
    Manager manager = getManager();
    if (manager != null) {
        manager.backgroundProcess();
    }

    //周期性的检查静态资源是否有变化
    WebResourceRoot resources = getResources();
    if (resources != null) {
        resources.backgroundProcess();
    }

    //调用父类ContainerBase的backgroundProcess方法
    super.backgroundProcess();
}
```

从上面的代码我们看到Context容器通过WebappLoader来检查类文件是否有更新，通过Session管理器来检

仅供个人学习 请勿传播 用户标记 mannix小查理

查是否有Session过期，并且通过资源管理器来检查静态资源是否有更新，最后还调用了父类ContainerBase的backgroundProcess方法。

这里我们要重点关注，WebappLoader是如何实现热加载的，它主要是调用了Context容器的reload方法，而Context的reload方法比较复杂，总结起来，主要完成了下面这些任务：

1. 停止和销毁Context容器及其所有子容器，子容器其实就是Wrapper，也就是说Wrapper里面Servlet实例也被销毁了。
2. 停止和销毁Context容器关联的Listener和Filter。
3. 停止和销毁Context下的Pipeline和各种Valve。
4. 停止和销毁Context的类加载器，以及类加载器加载的类文件资源。
5. 启动Context容器，在这个过程中会重新创建前面四步被销毁的资源。

在这个过程中，类加载器发挥着关键作用。一个Context容器对应一个类加载器，类加载器在销毁的过程中会把它加载的所有类也全部销毁。Context容器在启动过程中，会创建一个新的类加载器来加载新的类文件。

在Context的reload方法里，并没有调用Session管理器的distroy方法，也就是说这个Context关联的Session是没有销毁的。你还需要注意的是，Tomcat的热加载默认是关闭的，你需要在conf目录下的Context.xml文件中设置reloadable参数来开启这个功能，像下面这样：

```
<Context reloadable="true"/>
```

Tomcat热部署

我们再来看看热部署，热部署跟热加载的本质区别是，热部署会重新部署Web应用，原来的Context对象会整个被销毁掉，因此这个Context所关联的一切资源都会被销毁，包括Session。

那么Tomcat热部署又是由哪个容器来实现的呢？应该不是由Context，因为热部署过程中Context容器被销毁了，那么这个重担就落在Host身上了，因为它是Context的父容器。

跟Context不一样，Host容器并没有在backgroundProcess方法中实现周期性检测的任务，而是通过监听器HostConfig来实现的，HostConfig就是前面提到的“周期事件”的监听器，那“周期事件”达到时，HostConfig会做什么事呢？

```
public void lifecycleEvent(LifecycleEvent event) {
    // 执行check方法。
    if (event.getType().equals(Lifecycle.PERIODIC_EVENT)) {
        check();
    }
}
```

它执行了check方法，我们接着来看check方法里做了什么。

```
protected void check() {

    if (host.getAutoDeploy()) {
        // 检查这个Host下所有已经部署的Web应用
        DeployedApplication[] apps =
            deployed.values().toArray(new DeployedApplication[0]);

        for (int i = 0; i < apps.length; i++) {
            //检查Web应用目录是否有变化
            checkResources(apps[i], false);
        }

        //执行部署
        deployApps();
    }
}
```

其实HostConfig会检查webapps目录下的所有Web应用：

- 如果原来Web应用目录被删掉了，就把相应Context容器整个销毁掉。
- 是否有新的Web应用目录放进来了，或者有新的WAR包放进来了，就部署相应的Web应用。

因此HostConfig做的事情都是比较“宏观”的，它不会去检查具体类文件或者资源文件是否有变化，而是检查Web应用目录级别的变化。

本期精华

今天我们学习Tomcat的热加载和热部署，它们的目的都是在不重启Tomcat的情况下实现Web应用的更新。

热加载的粒度比较小，主要是针对类文件的更新，通过创建新的类加载器来实现重新加载。而热部署是针对整个Web应用的，Tomcat会将原来的Context对象整个销毁掉，再重新创建Context容器对象。

热加载和热部署的实现都离不开后台线程的周期性检查，Tomcat在基类ContainerBase中统一实现了后台线程的处理逻辑，并在顶层容器Engine启动后台线程，这样子容器组件甚至各种通用组件都不需要自己去创建后台线程，这样的设计显得优雅整洁。

课后思考

为什么Host容器不通过重写backgroundProcess方法来实现热部署呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。

仅供个人学习 请勿传播 用户标记 mannix小童鞋

极客时间

深入拆解 Tomcat & Jetty

从源码角度深度探索 Java 中间件



李号双

eBay 技术主管

新版升级：点击「请朋友读」，20位好友免费读，邀请订阅更有现金奖励。